

Boletín 02 Intermedio (Resuelto) — Primeros Pasos con Java

Soluciones completas, verificadas y comentadas línea a línea en Java 21 / 25 LTS con IntelliJ IDEA para los 13 ejercicios del Boletín Intermedio de la Unidad 2.

A continuación se presentan las soluciones comentadas a los 13 ejercicios propuestos en el [Boletín Intermedio](#). Cada solución ha sido optimizada y probada con Java 21 / 25 LTS en IntelliJ IDEA.

Solución al Ejercicio 1: Desglose Óptimo de Billetes y Monedas (Cajero Automático)

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class CajeroAutomatico {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce la cantidad en euros a retirar (entero): ");
        int cantidad = sc.nextInt();

        if (cantidad ≤ 0) {
            System.out.println("Error: La cantidad a retirar debe ser positiva.");
            sc.close();
            return;
        }

        System.out.printf("%nDispensando %d € con el mínimo número de piezas:%n",
cantidad);
        System.out.println("-----");

        int restante = cantidad;

        // Billetes de 50 €
        int b50 = restante / 50;
        restante %= 50;
        if (b50 > 0) System.out.printf("→ Billetes de 50 €: %d%n", b50);

        // Billetes de 20 €
        int b20 = restante / 20;
        restante %= 20;
        if (b20 > 0) System.out.printf("→ Billetes de 20 €: %d%n", b20);

        // Billetes de 10 €
        int b10 = restante / 10;
        restante %= 10;
        if (b10 > 0) System.out.printf("→ Billetes de 10 €: %d%n", b10);

        // Billetes de 5 €
        int b5 = restante / 5;
        restante %= 5;
        if (b5 > 0) System.out.printf("→ Billetes de 5 €: %d%n", b5);

        // Monedas de 2 €
        int m2 = restante / 2;
        restante %= 2;
        if (m2 > 0) System.out.printf("→ Monedas de 2 €: %d%n", m2);

        // Monedas de 1 €
        int m1 = restante;
        if (m1 > 0) System.out.printf("→ Monedas de 1 €: %d%n", m1);

        System.out.println("-----");
        sc.close();
    }
}

```

Solución al Ejercicio 2: El Envasado de Huevos de la Granja Avícola

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class EnvasadoGranja {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce el total de huevos recolectados hoy: ");
        int totalHuevos = sc.nextInt();

        if (totalHuevos < 0) {
            System.out.println("Error: La cantidad de huevos no puede ser negativa.");
            sc.close();
            return;
        }

        int restante = totalHuevos;

        // Envases gigantes: 144 huevos (1 gruesa)
        int envasesGrandes = restante / 144;
        restante %= 144;

        // Envases medianos: 12 huevos (1 docena)
        int docenas = restante / 12;
        restante %= 12;

        // Envases pequeños: 6 huevos (media docena)
        int mediasDocenas = restante / 6;
        restante %= 6;

        int sueltos = restante;

        System.out.println("\n" + "=".repeat(45));
        System.out.println("          INFORME DE CLASIFICACIÓN AVÍCOLA          ");
        System.out.println("=".repeat(45));
        System.out.printf("Total huevos procesados:           %d\n", totalHuevos);
        System.out.printf("Envases gigantes (144 ud):           %d\n", envasesGrandes);
        System.out.printf("Envases docenas   (12 ud):           %d\n", docenas);
        System.out.printf("Envases medias docenas (6 ud):       %d\n", mediasDocenas);
        System.out.printf("Huevos sueltos sin empaquetar:       %d\n", sueltos);
        System.out.println("=".repeat(45));

        sc.close();
    }
}
```

Solución al Ejercicio 3: Teorema de Pitágoras con Validación Dinámica

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class TeoremaPitagoras {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce la longitud del cateto 1 (m): ");
        double c1 = sc.nextDouble();

        System.out.print("Introduce la longitud del cateto 2 (m): ");
        double c2 = sc.nextDouble();

        if (c1 ≤ 0 || c2 ≤ 0) {
            System.out.println("Error: Los catetos de un triángulo deben ser estrictamente mayores que cero.");
            sc.close();
            return;
        }

        // Método 1: Raíz cuadrada de la suma de cuadrados
        double hipotenusa1 = Math.sqrt(Math.pow(c1, 2) + Math.pow(c2, 2));

        // Método 2: Función nativa optimizada de la clase Math
        double hipotenusa2 = Math.hypot(c1, c2);

        System.out.println("\n-----");
        System.out.printf("Cateto A:           %.3f m%n", c1);
        System.out.printf("Cateto B:           %.3f m%n", c2);
        System.out.printf("Hipotenusa (Math.sqrt): %.3f m%n", hipotenusa1);
        System.out.printf("Hipotenusa (Math.hypot): %.3f m%n", hipotenusa2);
        System.out.println("-----");

        sc.close();
    }
}

```

Solución al Ejercicio 4: Analizador y Parser de Nombres y Apellidos con String

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class ParserNombreCompleto {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce tu nombre y dos apellidos (separados por espacios):");

        String linea = sc.nextLine().trim();

        // Localizamos los dos espacios delimitadores clave
        int primerEspacio = linea.indexOf(' ');
        int ultimoEspacio = linea.lastIndexOf(' ');

        if (primerEspacio == -1 || primerEspacio == ultimoEspacio) {
            System.out.println("Error: Debes introducir un nombre y dos apellidos separados exactamente por espacios.");
            sc.close();
            return;
        }

        // Extracción de las 3 partes
        String nombre = linea.substring(0, primerEspacio);
        String apellido1 = linea.substring(primerEspacio + 1, ultimoEspacio);
        String apellido2 = linea.substring(ultimoEspacio + 1);

        // Iniciales
        char inNombre = Character.toUpperCase(nombre.charAt(0));
        char inApe1 = Character.toUpperCase(apellido1.charAt(0));
        char inApe2 = Character.toUpperCase(apellido2.charAt(0));
        String iniciales = "" + inNombre + inApe1 + inApe2;

        System.out.println("\n--- RESULTADOS DEL ANÁLISIS DE TEXTO ---");
        System.out.printf("Nombre:          %-15s (%d letras)%n", nombre.toUpperCase(), nombre.length());
        System.out.printf("Primer apellido: %-15s (%d letras)%n", apellido1.toUpperCase(), apellido1.length());
        System.out.printf("Segundo apellido: %-15s (%d letras)%n", apellido2.toUpperCase(), apellido2.length());
        System.out.printf("Iniciales:          %s%n", iniciales);

        sc.close();
    }
}

```

Solución al Ejercicio 5: Conversor de Potencia de Motores (CV a kW)

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class ConversorPotenciaMotores {
    private static final double KW_POR_CV = 0.73549875;
    private static final double CV_POR_KW = 1.35962;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("≡ CONVERSION DE POTENCIA DE VEHÍCULOS ≡");
        System.out.println("1. Convertir de Caballos de Vapor (CV) a Kilovatios (kW)");
        System.out.println("2. Convertir de Kilovatios (kW) a Caballos de Vapor (CV)");
        System.out.print("Selecciona una opción (1-2): ");
        int opcion = sc.nextInt();

        if (opcion == 1) {
            System.out.print("Introduce la potencia en CV: ");
            double cv = sc.nextDouble();
            if (cv ≤ 0) {
                System.out.println("Error: La potencia debe ser positiva.");
            } else {
                double kw = cv * KW_POR_CV;
                System.out.printf("%.2f CV equivalen exactamente a %.2f kW\n", cv, kw);
            }
        } else if (opcion == 2) {
            System.out.print("Introduce la potencia en kW: ");
            double kw = sc.nextDouble();
            if (kw ≤ 0) {
                System.out.println("Error: La potencia debe ser positiva.");
            } else {
                double cv = kw * CV_POR_KW;
                System.out.printf("%.2f kW equivalen exactamente a %.2f CV\n", kw, cv);
            }
        } else {
            System.out.println("Opción no válida.");
        }

        sc.close();
    }
}
```

Solución al Ejercicio 6: Tarificación Eléctrica con Tipos Enumerados (`enum`)

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class FacturacionElectrica {
    public enum TramoHorario {
        VALLE(0.11),
        LLANO(0.18),
        PUNTA(0.28);

        private final double precioKwh;

        TramoHorario(double precioKwh) {
            this.precioKwh = precioKwh;
        }

        public double getPrecioKwh() {
            return precioKwh;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.print("Introduce los kWh consumidos: ");
        double consumo = sc.nextDouble();

        System.out.print("Introduce el tramo horario (VALLE / LLANO / PUNTA): ");
        String tramoTexto = sc.next().toUpperCase();

        TramoHorario tramo;
        try {
            tramo = TramoHorario.valueOf(tramoTexto);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: Tramo desconocido. Opciones válidas: VALLE, LLANO o PUNTA.");
            sc.close();
            return;
        }

        double costeEnergia = consumo * tramo.getPrecioKwh();
        double impuestoElectrico = costeEnergia * 0.0511; // 5.11%
        double baseImponible = costeEnergia + impuestoElectrico;
        double iva = baseImponible * 0.21; // 21%
        double totalFactura = baseImponible + iva;

        System.out.println("\n" + "=".repeat(45));
        System.out.println("                        FACTURA DE ELECTRICIDAD                        ");
        System.out.println("=".repeat(45));
        System.out.printf("Tramo seleccionado:           %s (%.2f €/kWh)%n", tramo,
            tramo.getPrecioKwh());
        System.out.printf("Consumo registrado:           %.2f kWh%n", consumo);
        System.out.printf("Término de energía:          %8.2f €%n", costeEnergia);
        System.out.printf("Impuesto eléctrico (5.11%):%7.2f €%n", impuestoElectrico);
        System.out.printf("Base imponible:               %8.2f €%n", baseImponible);
        System.out.printf("IVA (21%):                    %8.2f €%n", iva);
        System.out.println("-".repeat(45));
        System.out.printf("TOTAL A PAGAR:                %8.2f €%n", totalFactura);
        System.out.println("=".repeat(45));
    }
}

```

```

        sc.close();
    }
}

```

Solución al Ejercicio 7: Clasificador de Caracteres Unicode y Métodos de `Character`

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class ClasificadorCaracteres {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Pulsa una tecla y presiona Enter: ");
        String entrada = sc.nextLine();

        if (entrada.isEmpty()) {
            System.out.println("No se ha introducido ningún carácter.");
            sc.close();
            return;
        }

        char c = entrada.charAt(0);
        int codigoAscii = (int) c;

        System.out.println("\n--- ANÁLISIS DE CARÁCTER ---");
        System.out.printf("Carácter analizado: '%c' (Código Unicode/ASCII: %d)%n", c,
            codigoAscii);

        if (Character.isDigit(c)) {
            System.out.printf("→ Tipo: DÍGITO NUMÉRICO (Valor entero: %d)%n",
                Character.getNumericValue(c));
        } else if (Character.isLetter(c)) {
            String caso = Character.isUpperCase(c) ? "MAYÚSCULA" : "MINÚSCULA";
            char normalizada = Character.toLowerCase(c);
            boolean esVocal = "aeiouáéíóúü".indexOf(normalizada) != -1;
            String subtipo = esVocal ? "VOCAL" : "CONSONANTE";
            System.out.printf("→ Tipo: LETRA DEL ALFABETO (%s, %s)%n", caso, subtipo);
        } else if (Character.isWhitespace(c)) {
            System.out.println("→ Tipo: ESPACIO EN BLANCO / TABULADOR / SALTO DE
                LÍNEA");
        } else {
            System.out.println("→ Tipo: SÍMBOLO O SIGNO DE PUNTUACIÓN");
        }

        sc.close();
    }
}

```

Solución al Ejercicio 8: Evaluación de Expediente con Expresión `switch` y `yield`

```
package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class EvaluacionExpedienteYield {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Introduce la calificación entera del expediente (1-10): ");
        int nota = sc.nextInt();

        // Expresión switch que utiliza yield en bloques multilínea
        String informe = switch (nota) {
            case 1, 2, 3, 4 → {
                System.out.println("[AUDITORÍA] Calificación por debajo del corte
legal.");
                yield "Insuficiente - Debe presentarse a la convocatoria
extraordinaria.";
            }
            case 5, 6 → {
                System.out.println("[AUDITORÍA] Supera las competencias básicas.");
                yield "Suficiente / Bien - Supera los mínimos exigidos en el módulo.";
            }
            case 7, 8 → {
                System.out.println("[AUDITORÍA] Rendimiento notable constatado.");
                yield "Notable - Demuestra un sólido dominio técnico de Java.";
            }
            case 9, 10 → {
                System.out.println("[AUDITORÍA] Rendimiento excelente de alta
graduación.");
                yield "Sobresaliente - Candidato a Matrícula de Honor.";
            }
            default → {
                System.out.println("[AUDITORÍA] Calificación anómala detectada.");
                yield "Nota no válida - El valor debe encontrarse en el rango 1 a 10.";
            }
        };

        System.out.printf("%nResultado del informe oficial:%n%s%n", informe);
        sc.close();
    }
}
```

Solución al Ejercicio 9: Calculadora de Edad Exacta con `java.time`

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.time.LocalDate;
import java.time.Period;

public class CalculadoraEdadExacta {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("≡≡≡ CALCULADORA DE EDAD CRONOLÓGICA EXACTA ≡≡≡");
        System.out.print("Introduce tu año de nacimiento (ej. 2004): ");
        int anio = sc.nextInt();

        System.out.print("Introduce tu mes de nacimiento (1-12): ");
        int mes = sc.nextInt();

        System.out.print("Introduce tu día de nacimiento (1-31): ");
        int dia = sc.nextInt();

        LocalDate nacimiento;
        try {
            nacimiento = LocalDate.of(anio, mes, dia);
        } catch (Exception e) {
            System.out.println("Error: La fecha indicada no es válida en el
calendario.");
            sc.close();
            return;
        }

        LocalDate hoy = LocalDate.now();

        if (nacimiento.isAfter(hoy)) {
            System.out.println("Error: ¡Aún no has nacido! La fecha de nacimiento es
posterior a hoy.");
            sc.close();
            return;
        }

        // Period calcula la diferencia exacta en años, meses y días
        Period edad = Period.between(nacimiento, hoy);

        System.out.println("\n-----");
        System.out.printf("Fecha de nacimiento: %s\n", nacimiento);
        System.out.printf("Fecha actual: %s\n", hoy);
        System.out.printf("Tu edad exacta es: %d años, %d meses y %d días.\n",
            edad.getYears(), edad.getMonths(), edad.getDays());

        // Comprobación de cumpleaños hoy
        if (nacimiento.getMonthValue() == hoy.getMonthValue() &&
nacimiento.getDayOfMonth() == hoy.getDayOfMonth()) {
            System.out.println("🎂🎉 ¡FELICIDADES! ¡Hoy es el día de tu cumpleaños!");
        }
        System.out.println("-----");

        sc.close();
    }
}

```

Solución al Ejercicio 10: Laboratorio de Desbordamiento Numérico (*Overflow*) y Casting

```
package es.iesperenxisa.primerospasos;

public class LaboratorioOverflow {
    public static void main(String[] args) {
        System.out.println("≡ LABORATORIO DE OVERFLOW Y CASTING ≡");

        // 1. Desbordamiento de 8 bits al convertir int a byte
        int entero = 130;
        byte byteForzado = (byte) entero;

        System.out.println("1. Casting de int (130) a byte:");
        System.out.printf("    Valor int original: %d\n", entero);
        System.out.printf("    Valor tras (byte): %d\n", byteForzado);
        System.out.println("    → Explicación: Un byte tiene un rango de -128 a 127.");
        System.out.println("    El valor 130 supera el límite máximo en 3 unidades.");
        System.out.println("    Al dar la vuelta en complemento a dos: 127 → -128, -127, -126.\n");

        // 2. Desbordamiento en multiplicación de enteros de 32 bits
        int millon = 1_000_000;
        int productoErroneo = millon * millon; // 10^12 supera 2.147.483.647 (Integer.MAX_VALUE)

        // Solución correcta forzando evaluación en 64 bits (long)
        long productoCorrecto = (long) millon * millon;

        System.out.println("2. Multiplicación de 1.000.000 x 1.000.000:");
        System.out.printf("    Resultado erróneo con 'int' (32 bits): %d\n", productoErroneo);
        System.out.printf("    Resultado exacto con 'long' (64 bits): %d\n", productoCorrecto);
        System.out.println("    → Explicación: Al multiplicar dos 'int', Java calcula en 32 bits");
        System.out.println("    desbordando silenciosamente. Se debe promocionar al menos un operando a 'long'.");
    }
}
```

Solución al Ejercicio 11: Posición Relativa de Dos Circunferencias en el Plano

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class PosicionRelativaCircunferencias {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("=== POSICIÓN RELATIVA DE DOS CIRCUNFERENCIAS ===");
        System.out.print("Circunferencia 1 → Centro x1, y1 y Radio r1: ");
        double x1 = sc.nextDouble();
        double y1 = sc.nextDouble();
        double r1 = sc.nextDouble();

        System.out.print("Circunferencia 2 → Centro x2, y2 y Radio r2: ");
        double x2 = sc.nextDouble();
        double y2 = sc.nextDouble();
        double r2 = sc.nextDouble();

        if (r1 ≤ 0.0 || r2 ≤ 0.0) {
            System.out.println("Error: Los radios deben ser estrictamente mayores que
cero.");
            sc.close();
            return;
        }

        // Distancia euclídea entre centros: d = sqrt((x2 - x1)^2 + (y2 - y1)^2)
        double d = Math.hypot(x2 - x1, y2 - y1);
        double sumaRadios = r1 + r2;
        double difRadios = Math.abs(r1 - r2);

        System.out.println("\n-----");
        System.out.printf("Distancia entre centros (d): %.4f\n", d);
        System.out.printf("Suma de radios (r1 + r2):      %.4f\n", sumaRadios);
        System.out.printf("Diferencia de radios |r1-r2|: %.4f\n", difRadios);

        String clasificacion;
        if (d == 0.0 && r1 == r2) {
            clasificacion = "COINCIDENTES (Mismo centro y mismo radio)";
        } else if (d == 0.0) {
            clasificacion = "CONCÉNTRICAS (Mismo centro, distinto radio)";
        } else if (d > sumaRadios) {
            clasificacion = "EXTERIORES (No se tocan, d > r1 + r2)";
        } else if (d == sumaRadios) {
            clasificacion = "TANGENTES EXTERIORES (Se tocan en un punto exterior, d ==
r1 + r2)";
        } else if (d > difRadios && d < sumaRadios) {
            clasificacion = "SECANTES (Se cortan en dos puntos, |r1 - r2| < d < r1 +
r2)";
        } else if (d == difRadios) {
            clasificacion = "TANGENTES INTERIORES (Se tocan en un punto interior, d ==
|r1 - r2|)";
        } else {
            clasificacion = "INTERIORES NO CONCÉNTRICAS (Una dentro de otra sin
tocarse, d < |r1 - r2|)";
        }

        System.out.printf("→ Clasificación Geométrica: %s\n", clasificacion);
        System.out.println("-----");
    }
}

```

```
        sc.close();  
    }  
}
```

Solución al Ejercicio 12: Validador Integral de Fechas con Año Bisiesto Gregoriano

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;

public class ValidadorFechaCompleta {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== VALIDADOR INTEGRAL DE FECHAS ===");
        System.out.print("Introduce el día: ");
        int dia = sc.nextInt();

        System.out.print("Introduce el mes (1-12): ");
        int mes = sc.nextInt();

        System.out.print("Introduce el año: ");
        int anio = sc.nextInt();

        // 1. Validar año positivo
        if (anio ≤ 0) {
            System.out.printf("Fecha INVÁLIDA: El año %d debe ser estrictamente mayor
que cero.%n", anio);
            sc.close();
            return;
        }

        // 2. Validar rango de mes
        if (mes < 1 || mes > 12) {
            System.out.printf("Fecha INVÁLIDA: El mes %d no existe (debe ser entre 1 y
12).%n", mes);
            sc.close();
            return;
        }

        // 3. Determinar año bisiesto gregoriano
        boolean esBisiesto = (anio % 4 == 0 && anio % 100 ≠ 0) || (anio % 400 == 0);

        // 4. Determinar días máximos del mes
        int diasMaximos = switch (mes) {
            case 1, 3, 5, 7, 8, 10, 12 → 31;
            case 4, 6, 9, 11 → 30;
            case 2 → esBisiesto ? 29 : 28;
            default → 0;
        };

        // 5. Validar día
        if (dia < 1 || dia > diasMaximos) {
            if (mes == 2 && dia == 29 && !esBisiesto) {
                System.out.printf("Fecha INVÁLIDA: El año %d no es bisiesto, por lo que
febrero solo tiene 28 días.%n", anio);
            } else {
                System.out.printf("Fecha INVÁLIDA: El día %d no es válido para el mes
%d (máximo permitido: %d).%n", dia, mes, diasMaximos);
            }
        } else {
            System.out.printf("¡Fecha VÁLIDA!: %02d/%02d/%04d %s%n",
                dia, mes, anio, (esBisiesto ? "(Año Bisiesto)" : "(Año Ordinario)"));
        }

        sc.close();
    }
}

```

```
}
```

Solución al Ejercicio 13: Ecuación de la Recta entre Dos Puntos y Tabla de Valores Tabulada

```

package es.iesperenxisa.primerospasos;

import java.util.Scanner;
import java.util.Locale;

public class RectaEntrePuntos {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in).useLocale(Locale.US);

        System.out.println("≡ ECUACIÓN DE LA RECTA QUE UNE DOS PUNTOS ≡");
        System.out.print("Introduce coordenadas de P1 (x1 y1): ");
        double x1 = sc.nextDouble();
        double y1 = sc.nextDouble();

        System.out.print("Introduce coordenadas de P2 (x2 y2): ");
        double x2 = sc.nextDouble();
        double y2 = sc.nextDouble();

        System.out.println("\n-----");
        if (x1 == x2) {
            // Recta vertical: pendiente indefinida
            System.out.printf("Ambos puntos comparten la misma coordenada X = %.3f%n",
x1);

            System.out.printf("Ecuación de la recta vertical: x = %.3f%n", x1);
            System.out.println("La pendiente es infinita; no es posible tabular como
función y = f(x).");
        } else {
            // Pendiente: m = (y2 - y1) / (x2 - x1)
            double m = (y2 - y1) / (x2 - x1);
            // Ordenada en el origen: n = y1 - m * x1
            double n = y1 - (m * x1);

            System.out.printf("Pendiente (m):           %.3f%n", m);
            System.out.printf("Ordenada en origen (n):   %.3f%n", n);
            System.out.printf("Ecuación explícita:      y = %.3f·x + (%.3f)%n%n", m,
n);

            // Tabla de valores para x en [-1.0, -0.5, 0.0, 0.5, 1.0]
            System.out.println("          TABLA DE VALORES CALCULADA          ");
            System.out.println("-----");
            System.out.printf("%-10s | %-15s%n", "x", "y = m·x + n");
            System.out.println("-----");

            double[] valoresX = {-1.0, -0.5, 0.0, 0.5, 1.0};
            for (double vx : valoresX) {
                double vy = (m * vx) + n;
                if (vy == -0.0) vy = 0.0; // Normalización visual
                System.out.printf("    %.3f    |    %.3f%n", vx, vy);
            }
            System.out.println("-----");
        }

        sc.close();
    }
}

```